SECTION 1

Convolutional Layers

- **Local receptive fields:** Each neuron connects to a small spatial patch of the previous map, tiling the image so early layers detect edges and textures and deeper layers compose parts.
- **Filters and feature maps:** Shared kernels slide across the input; one kernel yields one feature map highlighting where that pattern activates; depth is the count of learned filters.
- **Hyperparameters:** Stride and padding (“same” vs “valid”) set output spatial size; stacking 3×3 convs often beats a single large kernel for parameter efficiency; memory grows with map size, depth, and batch.
- **Implementation:** Keras `Conv2D` and TensorFlow `tf.nn.conv2d` express the same idea: learned filters scanned across height and width while mixing input channels.

SECTION 2

Pooling Layers

- **Downsampling:** Max or average pooling shrinks spatial dimensions, adds translation tolerance, and cuts compute; global average pooling collapses each map to a scalar summary.
- **Role in the stack:** Alternating conv blocks and pool steps widen channel depth while shrinking height and width, feeding a final dense or fully convolutional head.

SECTION 3

CNN Architectures

- **Typical pattern:** Repeated Conv → activation → pool, then flattened or global-pooled dense classifier with dropout on Fashion-MNIST–scale demos.
- **Classics:** LeNet-5 (digit recognition), AlexNet (deep ReLU stack, local response normalization, dropout, augmentation), GoogLeNet/Inception (multi-branch modules), ResNet (skip connections ease training of very deep nets), Xception (depthwise separable convolutions), plus SENet-style channel attention recalibration.
- **Practice:** Prefer smaller kernels, more depth, data augmentation over brute-force giant fully connected front ends on pixels.

SECTION 4

Using Pretrained Models for Transfer Learning

- **`keras.applications`:** One-line ImageNet weights for ResNet, Xception, MobileNet, etc.; call `preprocess_input` and match expected input sizes (often 224×224).
- **Fine-tuning flow:** Load with `include_top=False`, freeze base layers, add GlobalAveragePooling2D and a new softmax head, train the head, then unfreeze upper blocks with a lower learning rate on a domain dataset (e.g., TFDS flowers).
- **Augmentation:** Random crops and flips in `tf.data.map` or legacy `ImageDataGenerator` improve generalization when data are limited.

SECTION 5

Object Detection

- **Beyond single label:** Detection needs class plus spatial extent—many objects, many boxes per image.
- **Approaches sketched:** Sliding-window / region proposals with classifiers on crops; fully convolutional designs output grids of predictions; two-stage pipelines (feature CNN → region proposal network → refinement) balance accuracy and cost.
- **Metrics and data:** Training requires bounding-box labels (often the expensive annotation step); IoU measures overlap between predicted and ground-truth boxes.
- **Joint heads:** Models can emit class logits and box regressors together from a shared backbone, with weighted multi-task losses balancing label and localization error.

SECTION 6

Semantic Segmentation

- **Per-pixel classes:** Each pixel is labeled (e.g., road vs pedestrian); needs spatial resolution preserved rather than a single global vector.
- **Fully convolutional networks:** Replace dense layers with convolutions over feature maps so output tensors align spatially with inputs; upsampling (transpose convolutions) restores full resolution from a coarse encoding.
- **Design note:** Transposed convolutions learn how to expand feature maps but require careful stride and padding to avoid checkerboard artifacts; TensorFlow exposes standard conv and transpose ops for these graphs.